

DataLab 位运算实验报告

学号：2022201552 姓名：王建璋

摘要：本实验在课程服务器的 x86-64 Ubuntu 环境下完成 DataLab，在仅允许使用受限位运算算符的前提下实现 bits.c 中的 12 个函数，内容覆盖补码整数的判断、计数与位序操作，以及 IEEE 754 单精度与双精度浮点的位级构造与转换。报告依次说明实验环境与工具链、评测脚本的实际计分口径、最终得分与各函数的操作符用量，随后按整数类与浮点类分别给出实现思路，并归纳移位边界、TMin 取负与进位传递三个贯穿全实验的共性问题。最终 test.py 输出 Total points: 37/37，12 个函数在正确性与合规性上全部通过，编译过程无任何警告。

一、实验目的与实验环境

1.1 实验目的

- (1) 掌握补码表示下符号位、算术移位与掩码构造的关系，能在只有位运算的约束下完成符号判断、位计数与位序反转。
- (2) 理解 IEEE 754 单精度与双精度的字段划分，能够手工完成整数与浮点的相互转换、浮点乘 2 与 2 的幂次的位模式构造。
- (3) 掌握就近偶舍入规则，并在位级实现中正确处理尾数进位向阶码的传递。
- (4) 识别 C 语言中与位运算相关的未定义行为（移位量越界、有符号数溢出），并在实现中主动规避，而不是依赖某一编译器的实际表现。
- (5) 熟悉 make 构建、btest 正确性调试与 test.py 合规性评测构成的完整实验 workflow。

1.2 实验环境与工具链

本实验的编译、调试与评测全部在课程服务器上完成，本机仅作为编辑端：VS Code 通过 Remote-SSH 连接 ics-lab 主机，源码、构建产物与测试输出都位于远端，因此实验结果与本机的芯片架构无关。评测脚本所需的 pycparser 由仓库自带，无需额外安装。

表 1 实验环境与工具链

项目	配置
硬件	Intel Xeon Bronze 3204 @ 1.90GHz, 内存 2 GiB
操作系统	Ubuntu 22.04.5 LTS (x86-64)
编译器	gcc 11.4.0, 编译选项 -O0 -Wall -std=gnu99
构建与调试	GNU Make 4.3、GDB 12.1
评测脚本	Python 3.10.12, 配合仓库内置的 pycparser
开发环境	VS Code 1.138.0, 经 Remote-SSH 连接服务器
代码仓库	summerflower-kk/datalab2026 (fork 自 RUCICS2026/datalab2026)

二、实验要求与评测规则

2.1 编码约束

题目对每个函数分别给出了允许使用的算符集合 Legal ops 与算符总数上限 Max ops，除此之外还有若干全局约束：

- (1) 除题目明确允许外，禁止使用控制流语句、禁止调用任何函数、禁止使用宏。
- (2) 禁止定义数组、结构体等复合类型；float_i2f 之前的题目只能声明 int 变量，该函数及之后的浮点题目才可以声明 unsigned。
- (3) 禁止显式类型转换，也不得依赖未定义行为，编译不得产生任何警告。
- (4) 只允许修改 bits.c，且不得在其中添加头文件。

2.2 评测方式与实际计分口径

评测由 test.py 一次完成，分为正确性与合规性两部分：正确性部分编译并运行 btest，对每个函数在给定取值范围内与参考实现逐一比对；合规性部分用 pycparser 把 bits.c 解析成抽象语法树后做静态统计。两部分同时通过才计分，函数的分值等于题目标注的难度。

静态统计的口径与直觉有三处差别，实现时需要按它的规则来数：

第一，只有二元算符、一元算符和复合赋值算符计入总数，纯赋值 = 不计；复合赋值只算一次，例如 `v >>= s` 记作一个 `>>`，`r |= s` 记作一个 `|`。这一点直接影响 logtwo 这类算符紧张的题目的写法。

第二，if、while、for 等关键字以及变量的类型名只检查是否在白名单内，本身并不计入 Max ops，因此在允许的题目里使用分支不会挤占算符预算。

第三，任何显式类型转换都会被直接判为非法，数组同样非法，二者都不给部分分。

此外 decl.c 与 test.py 中记录的算符上限并不完全一致，例如 leftBitCount 在前者为 40、后者为 50，最终计分以 test.py 为准。

三、实验结果

3.1 总体得分

12 个函数的正确性与合规性全部通过，test.py 输出 Total points: 37，为满分；make 过程没有产生任何警告。图 1 为在 VS Code 集成终端中运行评测脚本的结果。

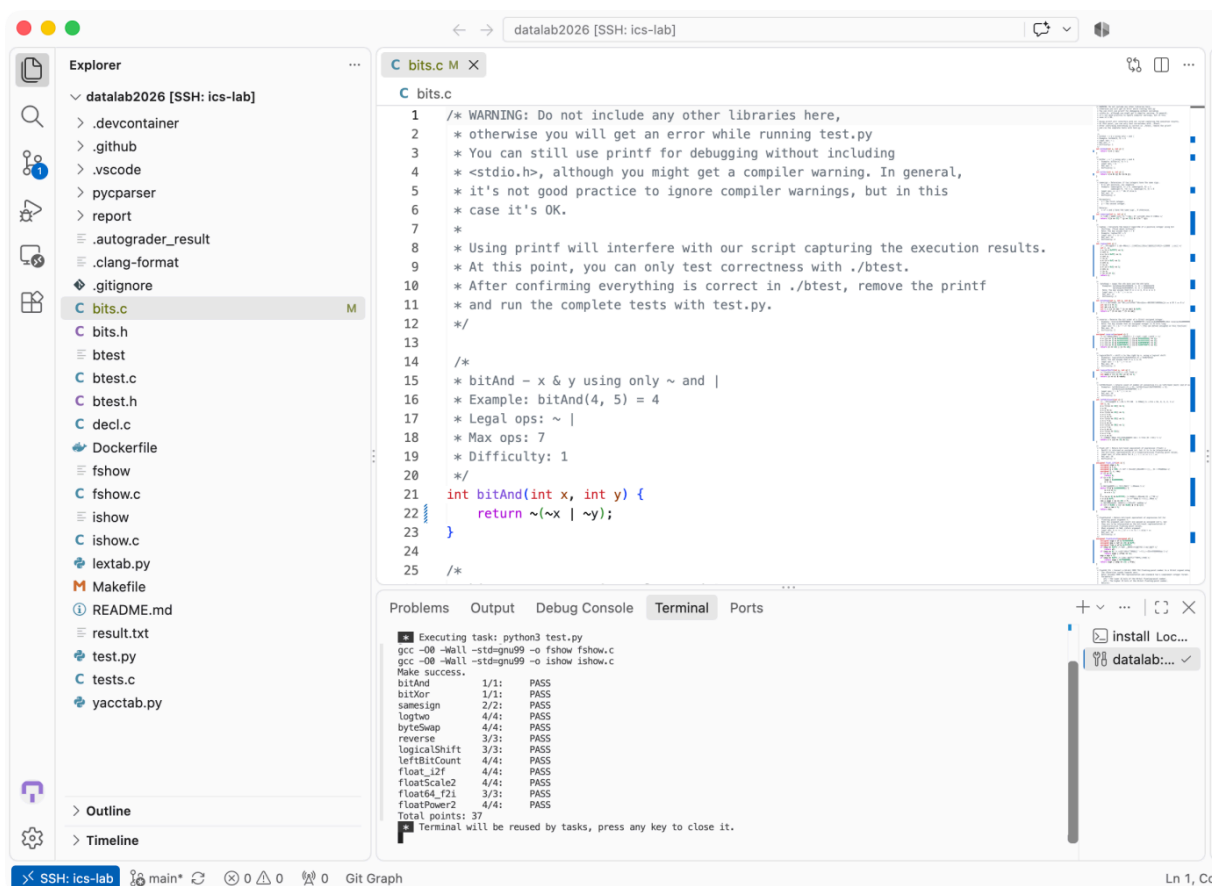


图 1 test.py 全部通过的运行结果（VS Code Remote-SSH 集成终端）

3.2 各函数得分与算符用量

表 2 汇总了每个函数的难度、得分、实际使用的算符数与上限，以及所用的主要手法。

表 2 各函数得分与算符用量

函数	难度	得分	算符数 / 上限	主要手法
bitAnd	1	1	4 / 7	德摩根律
bitXor	1	1	7 / 7	德摩根律（两次）
samesign	2	2	9 / 12	符号位与零值分开判断
logtwo	4	4	17 / 25	二分，以或代加累加位移量
byteSwap	4	4	10 / 17	异或的自反性
reverse	3	3	23 / 30	分治蝶形交换
logicalShift	3	3	6 / 20	由符号位构造高位掩码
leftBitCount	4	4	31 / 50	二分 + 全一检测
float_i2f	4	4	19 / 30	规格化 + 就近偶舍入
floatScale2	4	4	14 / 30	按阶码分类，只改阶码
float64_f2i	3	3	15 / 40	阶码定界 + 右移截断
floatPower2	4	4	9 / 30	直接拼装位模式
合计	—	37 / 37	—	—

3.3 实现亮点

按照算符用量相对上限的余量排序，下面四个函数的实现最为紧凑：

logicalShift (6 / 20) 、 floatPower2 (9 / 30) 、 byteSwap (10 / 17) 、 floatScale2 (14 / 30) 。

前两者的共同点是把问题整体转化为一次掩码构造或一次位模式拼装，完全避开逐位循环；byteSwap 利用异或的自反性，省去了「取出、清零、写回」三个步骤；floatScale2 则通过让非规格化数的进位自然落入阶码位，省去了边界特判。

四、整数类函数的实现思路

4.1 bitAnd 与 bitXor：德摩根律

bitAnd 只允许 ~ 和 |，直接套用德摩根律即可：

```
return ~(~x | ~y);
```

bitXor 只允许 ~ 和 &。异或可以写成「至少有一个为 1」且「不全为 1」，即 $(x | y) \& \sim(x \& y)$ ；其中的或运算再用一次德摩根律换成与运算：

```
return ~(~x & ~y) & ~(x & y);
```

该写法恰好使用 7 个算符，正好等于上限。

4.2 sameSign：符号位与零值分开判断

题目把 0 定义为既不为正也不为负，但又规定 sameSign(0, 0) 为 1，因此不能只比较符号位。把条件拆成互相独立的两部分后问题就清楚了：一是符号位相同，二是「是否为零」这一属性相同。

```
return !((x >> 31) ^ (y >> 31)) & !(~x ^ ~y);
```

只看第一个条件会把 (0, 5) 误判为同号，而第二个条件正好把它排除；对 (0, 0) 而言两个条件同时成立，结果为 1，与题目要求一致。共使用 9 个算符。

4.3 logTwo：仅用比较与移位的二分

$\text{floor}(\log_2(v))$ 就是 v 的最高有效位的下标。本题可用的算符只有 $> < >> << |$ ，既没有加法也没有与运算，因此采用二分，并用或运算完成累加：

```
r = (v > 0xFFFF) << 4; v >>= r;
s = (v > 0xFF) << 3; v >>= s; r |= s;
/* 对 0xF、0x3 再各做一轮 */
r |= (v >> 1);
```

这里有两个关键点。其一，比较运算的结果只能是 0 或 1，把它左移 k 位就直接得到了「本轮应当右移多少位」，省去了分支；其二，五轮的位移量分别取自 16、8、4、2、1，在二进制下互不重叠，因此可以用按位或代替加法完成累加。每一轮都把已经确认存在的高位移走，循环结束时 v 只剩一位或两位，最后一位答案由右移一位直接给出。共使用 17 个算符。

4.4 byteSwap: 利用异或的自反性

交换两个字节的常规做法是取出两个字节、把原位置清零、再写回对方的值，步骤多且算符消耗大。利用异或的自反性可以一步完成：设两个字节分别为 a 与 b ，令 $t = a \oplus b$ ，则在第 n 字节处异或 t 得到 $a \oplus (a \oplus b) = b$ ，在第 m 字节处异或 t 得到 $b \oplus (a \oplus b) = a$ 。

```
int t = ((x >> sn) ^ (x >> sm)) & 0xFF;
return x ^ (t << sn) ^ (t << sm);
```

其中 sn 、 sm 是两个字节的位偏移。 t 必须先与 $0xFF$ 相与截断为一个字节，否则移位后会污染相邻字节。这种写法还有一个好处：当 n 等于 m 时 t 恒为 0，两次异或都不改变原值，无需单独特判。共使用 10 个算符。

4.5 reverse: 分治的蝶形交换

整体位序反转可以分解为若干轮块内交换：先交换相邻的 1 位，再交换相邻的 2 位块，随后是 4 位、8 位、16 位，共 5 轮即可完成 32 位的反转。每一轮用一对互补的掩码把奇偶块分离，再分别左右移动后合并。

```
v = ((v >> 1) & 0x55555555) | ((v & 0x55555555) << 1);
/* 0x33333333、0x0F0F0F0F、0x00FF00FF 各做一轮 */
return (v >> 16) | (v << 16);
```

最后一轮交换高低各 16 位，移位之后不会有多余的位残留，因此不需要掩码，相比前四轮可以省下两个算符。共使用 23 个算符。

4.6 logicalShift: 掩码构造与移位边界

思路是先做算术右移，再把被符号位填充的高 n 位强行清零，难点在于构造这个掩码而又不触发未定义行为。

```
int mask = ((1 << 31) >> n) << 1;
return (x >> n) & ~mask;
```

$1 \ll 31$ 得到符号位；对它做算术右移 n 位，高 $n+1$ 位会被填成 1；再左移一位把多出来的那一位移出去，剩下的正好是高 n 位。取反后与算术右移的结果相与，即得到逻辑右移。这样构造的另一个好处是全程移位量都落在 0 到 31 之间：当 n 为 0 时掩码为 $0x80000000 \ll 1$ ，结果为 0，取反后是全 1，不需要用「右移 32 位」这种未定义行为来处理边界。共使用 6 个算符，是全部题目中余量最大的一个。

4.7 leftBitCount: 二分统计前导 1

与 `logtwo` 同属二分，只是判定条件换成「最高 k 位是否全为 1」。把 x 算术右移 $32 - k$ 位后，若结果为全 1，则说明最高 k 位都是 1；判定全 1 可以先取反再用逻辑非，即 $!(\sim(x \gg (32 - k)))$ 。条件成立时把计数加 k ，并把 x 左移 k 位，继续考察后面的位。

```
n = !(\sim(x >> 16)) << 4; c = n; x = x << n;
n = !(\sim(x >> 24)) << 3; c = c + n; x = x << n;
/* 对 28、30、31 再各做一轮 */
return c + ((x >> 31) & 1);
```

k 依次取 16、8、4、2、1 共五轮，能表示的最大计数是 31，而本题的答案最大为 32，因此循环之后还要补一次判断：如果此时的最高位仍然是 1，说明 32 位全为 1，计数再加 1。共使

用 31 个算符。

五、浮点类函数的实现思路

5.1 float_i2f: 规格化与就近偶舍入

整数转单精度浮点分三步：取绝对值、规格化、舍入。

第一步，把 x 赋给 unsigned 变量 a 后，对负数执行 $a = -a$ 。这里刻意使用无符号取负：当 x 为 TMin 时，在 int 下取负会溢出，而无符号取负有明确定义，其结果 $0x80000000$ 恰好就是 TMin 的真实幅值 2^{31} 。

第二步，把 a 左移到最高位为 1。阶码初值取 158，即 $127 + 31$ ，每左移一位减 1——因为规格化之后隐含的那一位固定落在第 31 位，左移的次数正好等于阶码相对初值的减少量。

第三步，第 30 位至第 8 位构成 23 位尾数，被移出的低 8 位用于舍入。按就近偶舍入规则：余数大于 $0x80$ 时进位；等于 $0x80$ 且尾数最低位为 1（即需要向偶数取整）时也进位；其余情况舍去。

```
if ((r > 0x80) | ((r == 0x80) & (f & 1)))
    res = res + 1;
```

进位直接加在已经拼好的 $\text{sign} \mid \text{exp} \mid \text{frac}$ 上。这样做的好处是：若尾数因进位而溢出，多出来的 1 会自然传入阶码字段，正好对应阶码加一、尾数归零的结果，不需要另写分支处理。共使用 19 个算符。

5.2 floatScale2: 按阶码分类，只改阶码

浮点数乘 2 按阶码分成三种情况，三者都不需要改动尾数的有效数字：

第一，阶码为 $0xFF$ 时，NaN 按题目要求原样返回，无穷乘 2 仍是无穷、同样原样返回，因此可以合并成一条分支。

第二，阶码为 0 时是非规格化数，把尾数左移一位即可。若尾数因此溢出，进位会自动落进阶码字段使其变为 1，而这正是「最大的非规格化数翻倍后成为最小的规格化数」的正确结果，因此不必特判这个边界。

第三，其余情况阶码加 1；若加完之后等于 $0xFF$ 说明上溢，此时必须返回无穷，尾数要清零，而不能把原尾数拼回去——否则得到的是 NaN 而非无穷。共使用 14 个算符。

5.3 float64_f2i: 阶码定界与右移截断

双精度的阶码位于 uf2 的第 30 位至第 20 位。先用阶码把两类越界情况挡在外面：阶码小于 1023 时 $|x| < 1$ ，向零舍入得 0；阶码大于 1054 时 $|x| \geq 2^{32}$ ，必然越界，NaN 与无穷也落在这一类。

```
val = 0x80000000 | ((uf2 & 0xFFFFF) << 11) | (uf1 >> 21);
val = val >> (1054 - exp);
```

中间区间把隐含的 1 补到第 31 位，再拼上 uf2 的低 20 位与 uf1 的高 11 位，凑成 32 位有效数字，然后右移 $1054 - \text{exp}$ 位。这样设计有两个好处：移位量恰好落在 0 到 31 之间，不会越界；右移本身就是向零截断，与题目要求的舍入方式一致，不必额外处理舍入。

最后按符号判断是否越界。需要注意正负两侧的阈值并不对称：正数的上界是 `0x7FFFFFFF`，而负数可以取到 `0x80000000`（即 TMin），因此两个分支要分别比较。共使用 15 个算符。

5.4 floatPower2：直接拼装位模式

2.0 的 x 次幂无需真的做乘法，其位模式可以按 x 所在的区间直接拼出来：

第一，当 $-126 \leq x \leq 127$ 时结果是规格化数，尾数全为 0，阶码为 $x + 127$ ，故返回 $(x + 127) \ll 23$ 。

第二，当 $-149 \leq x \leq -127$ 时结果是非规格化数，其值等于尾数乘以 2 的 -149 次幂，因此尾数应为 $1 \ll (x + 149)$ 。

第三， x 小于 -149 时返回 0， x 大于 127 时返回正无穷。

实现上按 $x > 127$ 、 $x < -149$ 、 $x < -126$ 的顺序写三个判断，落到最后一条返回语句的自然就是规格化区间。 x 等于 TMin 时会被第二个条件拦下并返回 0，无需单独判。共使用 9 个算符。

六、关键问题分析

6.1 移位量的边界

本实验中最容易出错的地方是移位量取到 0 或 32。C 标准规定移位量大于等于操作数位数时行为未定义，题目也明确禁止依赖这类行为，因此不能用「右移 32 位得到 0」之类的假设。

解决办法是把掩码构造成只依赖 0 到 31 范围内移位的形式。以 `logicalShift` 为例，若直接构造「低 $32 - n$ 位为 1」的掩码， n 为 0 时就需要移位 32 位；而改为从符号位出发、先算术右移 n 位再左移一位， n 为 0 时对应的是 `0x80000000 << 1`，移位量仍在合法范围内，结果为 0，取反后恰好是全 1。同理，`float64_f2i` 中把有效数字对齐到第 31 位，使实际移位量恒为 $1054 - \text{exp}$ ，配合前面的阶码定界后必然落在 0 到 31 之间。

6.2 TMin 取负的溢出

`float_i2f` 需要整数的绝对值，但 TMin 的相反数不在 `int` 的表示范围内， $-x$ 与 $\sim x + 1$ 在 `int` 下都会溢出。把变量声明为 `unsigned` 后再取负，运算按模 2^{32} 进行，结果 `0x80000000` 正是 2^{31} ，与 TMin 的真实幅值一致，既避免了溢出又不需要对 TMin 单独分支。

这一处理也说明了一个更一般的经验：当需要的中间量超出有符号类型的范围时，换用同宽度的无符号类型往往比增加特判更简洁，并且语义上有明确定义。

6.3 进位的自然传递

浮点题目中有两处可以借助字段的相邻关系省去分支。一是 `float_i2f` 的舍入进位：因为阶码字段紧邻尾数字段的高位，把进位加在完整的位模式上时，尾数溢出会自动进位到阶码，恰好对应正确结果。二是 `floatScale2` 中非规格化数的处理：尾数左移溢出时进位落入阶码，自动完成从非规格化到规格化的过渡。

但这种「自然传递」并非处处适用。`floatScale2` 在阶码加一后变为 `0xFF` 时，如果仍按统一公式拼回原尾数，得到的会是 NaN 而不是无穷，必须单独把尾数清零。判断某个边界能否依

靠位模式自然过渡，关键在于该边界两侧的编码是否在数值上连续。

七、实验总结

本次实验完整地走过了从阅读题目约束、设计位级算法、用 btest 调试正确性，到用 test.py 检查合规性的全过程，12 个函数全部通过，得分 37/37。

整数部分的题目大多可以归结为两个套路：一是二分定位，用于寻找最高有效位或统计前导 1，其代价是对数级而非线性级；二是掩码构造，把「保留哪些位、清除哪些位」转化为一次与运算。掌握这两个套路之后，真正花时间的反而不是主体逻辑，而是边界情况——移位量为 0、输入为 TMin、非规格化数与规格化数的交界。实践下来更有效的做法是在动手之前先把这些极端输入列成清单逐个推演，比写完之后靠 btest 反复试错要快得多。

浮点部分让我真正理解了 IEEE 754 编码设计的巧妙之处。阶码采用移码而非补码、隐含位的引入、非规格化数与规格化数在编码上的连续，这些设计使得很多操作可以退化为简单的位运算：乘 2 只需阶码加一，非规格化数翻倍靠尾数进位就能自动跨越边界，舍入进位可以直接加在完整位模式上。此外，评测脚本对算符的静态统计口径也带来了额外的约束。复合赋值只计一次、关键字不计入总数这些规则，直接影响了 logtwo 这类算符紧张题目的写法——把累加写成 `|=` 而不是 `+=`，既符合题目给定的算符集合，也节省了计数。